

The workflow, **as an engine**

What our task & workflow model does today, why founders lose trust in it, and the smallest rebuild that fixes it.

AUTHOR
Engineering

AUDIENCE
Founder · Product · Eng

STATUS
Draft for discussion

LENGTH
~10 min read

Two parallel worlds share a page and a name — **but never a lifecycle.**

That's the whole bug, dressed up in three different Settings cards.

↳ *Everything that follows is a symptom of this one gap.*

Tasks and workflows are strangers

TASK

- **A thing to do**
assignee · due date · status (todo → in_progress → done)
- **Has a story**
"Priya, call the vendor about the missing shipment"
- **Lives in tasks**

WORKFLOW

- **One instance of a business process**
one order, one purchase — moving across stages
- **Has a stage**
frozen stage list · title · amount · counterparty
- **Lives in workflows**

THE CONSEQUENCE

Advancing a stage doesn't create, assign, or close any task. Closing a task doesn't advance any workflow. They share only a `decision_id` when spawned from the same voice capture — nothing else touches them together.

01.2

Three cards, three shapes, one concept

Settings is one long scroll of eight cards. Three describe the same thing — three different ways, each with its own Save button.

CONCEPT	COMPANY DETAILS	BUSINESS VOCABULARY	OPERATING MODEL
Workflows	Free-text names → workflow_templates	Three fixed labels → lexicon.workflows	Full pipelines + approval gates → operating_model.pipelines
Task categories	Fixed enum	Six fixed labels	Dynamic list
Approvals	Free-text rules	—	Per-pipeline owner sign-off stage

All three feel authoritative; only **Operating Model** actually drives behaviour — and it's the least discoverable.

01.3

The Kapoor order — what happens now

"We got an order from Kapoor Retail for 500m cotton, ₹80k, deliver by Friday."

WHAT THE AI CREATES

- **Workflow card "Order — Kapoor Retail"**
placed at stage Booked in the Sales pipeline
- **Two tasks**
"Confirm order" (Sales) · "Prepare invoice" (Finance)

WHAT HAPPENS NEXT

- **The card just... sits at Booked**
until someone opens Workflows and clicks Advance
- **Tasks flow todo → done independently**
Priya closing hers never moves the card

The board and the work are two views of two different truths.

One lifecycle, **one view**

The workflow becomes the engine. Each stage owns the tasks and approvals that define what “being at this stage” means — completing them moves the card. The founder configures once and watches the business run itself.

A stage is a contract, not a label

Each stage declares four things:

01

Which tasks fire

On entry, tasks are created for the named roles and become the current work.

02

Who must approve

A role, a specific owner, or nobody. Skipping the gate is impossible.

03

What advances it

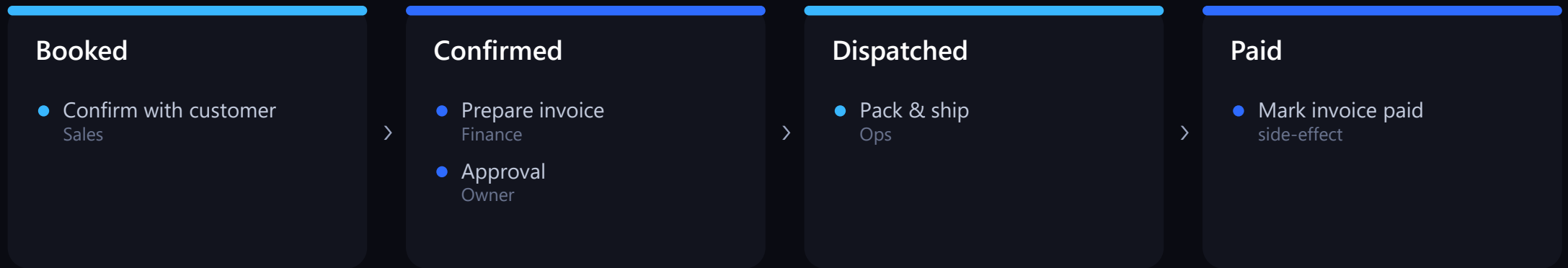
Default: all tasks done AND all approvals recorded. Override is audited.

04

What side-effects fire

On entry/exit — the procurement → awaiting-bill expense generalises to any stage.

Same capture — now the card owns the work



Tasks aren't siblings — they're **children** of the card. Priya closing “Confirm with customer” fires the transition check; when it passes the card moves to Confirmed and the next task appears in Finance's queue. My Work stops asking “which task next?” — it shows the tasks the pipeline is generating.

Three changes — everything else is deletion

TODAY

tasks — no link to a workflow

workflows.stages[] — a flat list of strings

tenant.workflow_templates — dead brainstorm list

lexicon.workflows — three hardcoded labels

operating_model.pipelines — real, but unlinked

PROPOSED

tasks.workflow_id · tasks.stage_key (every task knows its parent card + stage)

pipelines[].stages[] — objects: {key, label, tasks[], approval, side_effects[]}

removed

removed

The single source of truth

03.2

One module, three operations

[services/workflow_engine.py](#)

FUNCTION	FIRES WHEN	DOES
on_stage_enter	Card arrives at a new stage	Spawns the stage's tasks for named roles, records a timeline entry, dispatches notifications
check_stage_ready	A task closes, or an approval is recorded	True only if every stage task is done AND every required approval is present
advance	Task closer, approver, or manual button	On pass → transition + re-enter on_stage_enter. Terminal stage → fire side-effects, mark done

The engine is the only path that writes [workflows.stage](#). Manual advance goes through it too, with an audited override. No more silent kanban drags.

Three migrations — routed through the ledger

One-time · exactly-once · safe on replica restart

01

Backfill the links

Set `tasks.workflow_id` + `stage_key` via `decision_id`.
No match → fields stay null, task works as ad-hoc.

02

Convert the stages

Flat strings → objects
{`key`,`label`,`tasks`:[],`approval`:null}. Empty templates
preserve today's behaviour.

03

Drop the ghosts

Remove `workflow_templates` + `lexicon.workflows`.
Nothing writes them; no reader depends on them.

Eight scattered cards → four tabs

TAB	WHAT LIVES HERE
Business	Company profile, products, industry, team roles
Operations	Pipelines. Each shows its stages inline; each stage shows task templates + approval rule. The only place workflow config lives.
Money	Finance categories, high-value approval threshold, currency
Account	Language, profile, security

Operations tab — one hierarchy: **Pipeline** → **Stages** → **Tasks** + **Approval**. Add a task template inline; set an approval via dropdown.

04.2

My Work + Workflow, joined

TODAY

Tasks with no context about which card they belong to

Cards with no context about the tasks inside

Two views, two truths

PROPOSED

Every task carries a stage chip — “Order #4821 · Confirmed”. Click to open the parent card

Every card lists its current stage's open tasks inline, with the assignee's avatar

Same data, two lenses

The Advance button stays but is rarely needed — cards advance themselves when the stage contract is satisfied. A manual click asks for a reason, recorded in history, so the audit trail knows why the engine was overridden.

No big-bang — three valuable, reversible phases

01

~1 week

Unify the model

Kill the ghost collections, add `workflow_id` + `stage_key` with a backfill, ship the Settings tabs. Behaviour unchanged; confusion gone.

02

~2 weeks

Introduce the engine

Ship `workflow_engine.py`, migrate stages to objects, wire `on_stage_enter` + `advance`. Existing cards keep working; new cards use the engine as owners fill templates.

03

~1 week

The joined view

Stage chips on My Work, inline task lists on cards, manual advance needs a reason. Retire the redundant Board sub-tab.

What we're deliberately **not** doing

Each is real; none are needed to earn back trust in the board.

Cross-pipeline dependencies

SLA timers

Workload balancing

Forecast reports

Ship the engine, watch what happens, then decide what deserves the next investment.